

1. Suppose you have a counting semaphore S. After certain number of wait() and signal() operations, the value of S is -4. Determine the number of processes waiting on S. Assume semaphore implementation without busy waiting. You do not have to worry about how the value -4 was arrived upon.

1 point

- ☒ 4
- ☐ 3
- ☐ 1
- ☐ -4

2. Which of the following is true?

1 point

- ☐ Spinlock does not cause busy waiting.
- ☒ wait() and signal() should be executed atomically.
- ☐ Peterson's solution is a hardware-based solution.
- ☐ Semaphore implementation with busy waiting may cause the semaphore value to be negative.

3. In a typical program, which of the following code segments determines which process can next execute the critical section?

1 point

- ☒ entry section
- ☐ exit section
- ☐ remainder section
- ☐ mutual exclusion section

4. Consider the following solution to the critical section problem for a 2 process system. The 2 processes are P1 and P2. The solution uses two binary semaphores sem1 and sem2. Both the semaphores are initialized to 1. Value of both semaphores range only between 0 and 1. Assume semaphore implementation with busy waiting.

Pseudocode for P1:

```
signal(sem1);  
  
signal(sem2);  
  
//critical section  
  
wait(sem1);  
  
wait(sem2);
```

Pseudocode for P2:

```
wait(sem1);  
  
wait(sem2);  
  
//critical section  
  
signal(sem1);  
  
signal(sem2);
```

Which of the following is true?

- ☐ P2 starves
- ☒ Mutual exclusion is violated
- ☐ P1 and P2 will be deadlocked
- ☐ P1 starves

1 point

5. Consider the solution to the producer-consumer problem that you have learnt. Assume that there are multiple producer processes as well as multiple consumer processes executing simultaneously. If the value of the full semaphore is -3 and the value of the empty semaphore is -4, then calculate the total number of blocked processes. Assume semaphore implementation without busy waiting. You do not have to worry about how the values of full and empty semaphores were derived.

- ☐ 4
- ☐ 3
- ☐ -7
- ☒ 7